Module 5 — Assembly Language Machine Problems

NASM 32-bit | Procedures | Conditional Selection | User Input | ASCII Art

Two interactive console applications • Intermediate difficulty • Recommended time: 2–3 hours per problem

FINAL LABORATORY 2

Rhiane Alexey R. Pascua	**Raymond Jr. Ricohermoso**
TN24	CCS003L
Dr. Hadji Tejuco	FLAB2

General Programming Requirements

Develop both applications using NASM x86 32-bit assembly on Linux with int 0x80. Programs must compile and run without errors.

Requirement	Minimum
Procedures using CALL and RET	5
User input operations	3
Conditional selections	5
Menu options	4
ASCII art designs	3
Menu repetition or loop	1
Input validation and exit option	Required

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

Problem Statement

Create an interactive Zodiac Horoscope and Fortune Teller System. The user enters a name, birth month and birth day. The program identifies the zodiac sign and displays a fictional horoscope according to a selected category. Separate major tasks into procedures and use conditional branches for date and menu decisions.

1. Welcome Screen

```
========================================
       THE MYSTIC ZODIAC ORACLE
========================================
             *       *
          *     /\       *
               /  \
         *    /____\    *
              |  |
            *   |__|   *
       DISCOVER YOUR DESTINY!
========================================
```

2. User Information

```
Enter your name: Maria
Enter birth month (1-12): 8
Enter birth day (1-31): 15

Welcome, Maria!
```

Validate month and day against actual month lengths. February 29 may be rejected because no birth year is collected.

3. Zodiac Identification

Sign	Date range
Capricorn	Dec 22–Jan 19
Aquarius	Jan 20–Feb 18
Pisces	Feb 19–Mar 20
Aries	Mar 21–Apr 19
Taurus	Apr 20–May 20
Gemini	May 21–Jun 20
Cancer	Jun 21–Jul 22
Leo	Jul 23–Aug 22
Virgo	Aug 23–Sep 22
Libra	Sep 23–Oct 22
Scorpio	Oct 23–Nov 21
Sagittarius	Nov 22–Dec 21

Display the sign, element, symbol, personality description, and simplified ASCII art corresponding to the selected sign.

```
========================================
         YOUR ZODIAC SIGN
========================================
              /\_/\
             ( o.o )
              > ^ <
Zodiac Sign: LEO
Element: Fire
Symbol: Lion
Personality: Confident, creative, courageous.
```

4. Horoscope Menu

```
[1] Love and Relationships
[2] Career and Studies
[3] Health and Wellness
[4] Lucky Number and Color
[5] View Zodiac Information
[6] Exit
Enter your choice:
```

Choices 1–3 display category-specific fictional predictions; choice 4 displays a lucky number and color; choice 5 shows zodiac details and art; choice 6 exits. Invalid selections must display an error and return to the menu.

5. Sample Interaction

```
Enter your name: Maria
Enter birth month: 8
Enter birth day: 15
Hello, Maria!
```

```
Your Zodiac Sign: LEO
Your Element: FIRE
Choice: 2
CAREER HOROSCOPE
Leo, your creativity will help you
solve a difficult problem today.
Stay focused and confident!
Return to menu? (Y/N): Y
```

Required Procedures

Procedure	Responsibility
display_welcome	Display welcome ASCII art
get_user_info	Read name and birth date
validate_birthdate	Check date validity
determine_zodiac	Identify zodiac sign
display_zodiac_art	Display sign-specific ASCII art
display_menu	Display horoscope choices
show_horoscope	Select and show prediction
exit_program	Terminate program

Required branching: CMP, JE/JNE for menus, and JL/JG/JLE/JGE for date ranges; include a menu loop.

Test Cases

Test	Input	Expected result
1	March 21	Aries
2	April 19	Aries
3	April 20	Taurus
4	June 15	Gemini
5	July 23	Leo
6	August 15	Leo
7	December 25	Capricorn
8	February 30	Invalid date
9	Month 13	Invalid month
10	Menu choice 9	Invalid selection

Explanation of Implemented Procedure

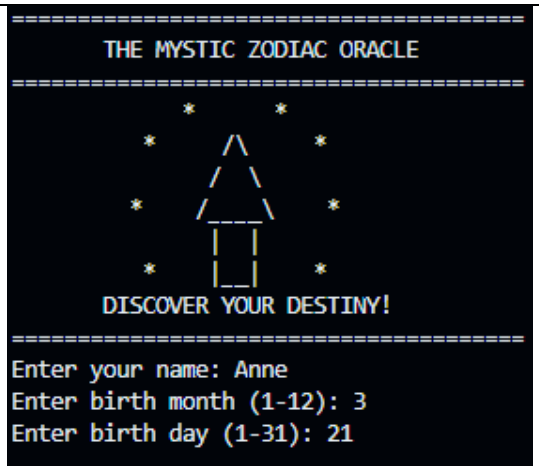
1. display_welcome
This procedure displays the welcome banner and ASCII art at the start of the program. It uses the Linux write system call (int 0x80) to print the title "THE MYSTIC ZODIAC ORACLE" along with the decorative design. Its purpose is to introduce the user to the horoscope system before any input is collected.

2. get_user_info
This procedure collects the user's name, birth month, and birth day. It reads the inputs from the keyboard, stores them in memory variables, and removes the newline character from the entered name. After gathering the information, it displays a personalized welcome message containing the user's name.

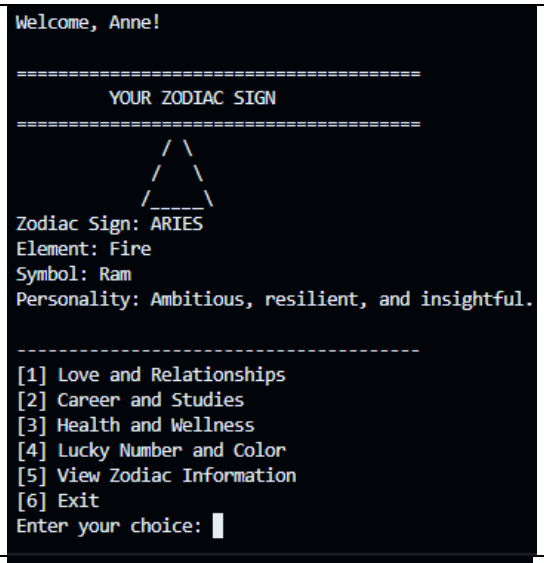
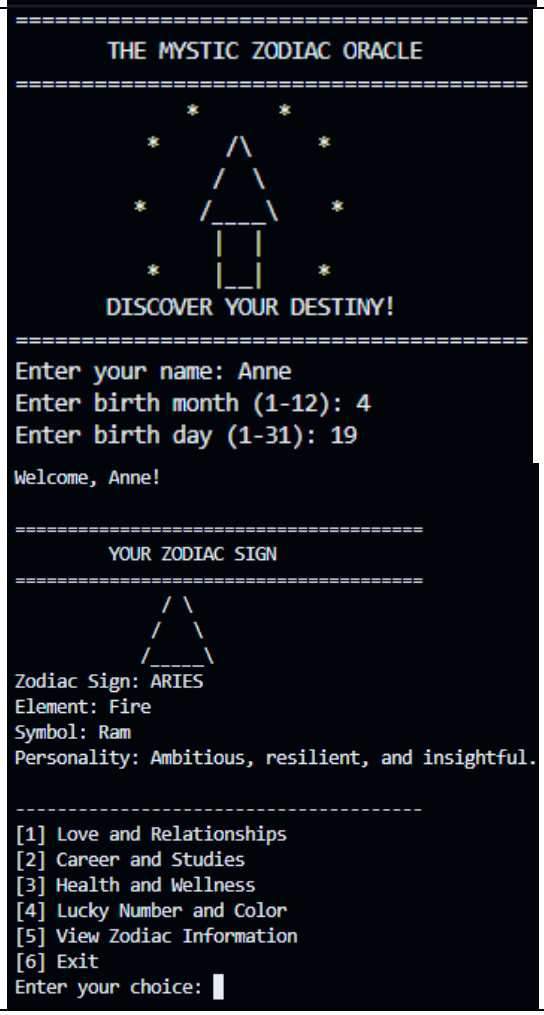
3. validate_birthdate
This procedure checks whether the entered month and day form a valid birth date. It

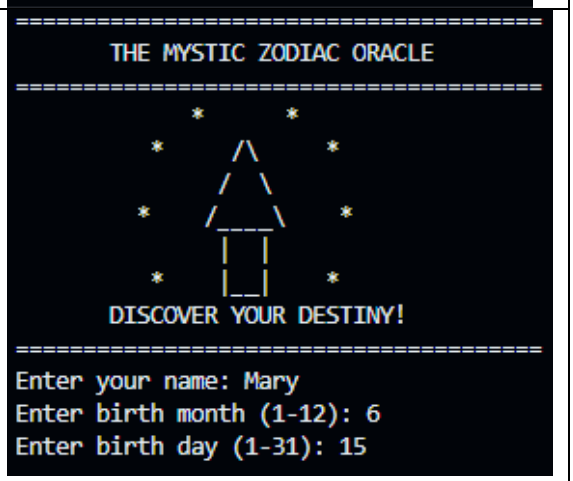
verifies that the month is within the range of 1–12 and ensures that the day does not exceed the allowed number of days for each month category, including February's 28-day limit. If the date is valid, it returns 1; otherwise, it returns 0.
4. determine_zodiac This procedure identifies the user's zodiac sign based on the entered birth month and day. It uses a series of comparisons and conditional jumps to determine which zodiac date range the birth date belongs to. The resulting zodiac sign is stored in the zodiac_id variable for later use.
5. display_zodiac_art This procedure displays the zodiac information to the user. It prints the zodiac sign, element, symbol, personality description, and corresponding ASCII art. The program shows a special lion ASCII art for Leo and a generic design for all other zodiac signs.
6. display_menu This procedure prints the horoscope menu options on the screen. It allows the user to choose from different categories such as Love and Relationships, Career and Studies, Health and Wellness, Lucky Number and Color, and Zodiac Information. The menu serves as the main navigation interface of the program.
7. show_horoscope This procedure processes the selected menu option and displays the corresponding horoscope content. It uses conditional comparisons to determine which prediction should be shown. Depending on the user's choice, it displays information related to love, career, health, lucky numbers, or zodiac details.
8. exit_program This procedure safely terminates the application. Before exiting, it displays a farewell message thanking the user for consulting the Zodiac Oracle. It then uses the Linux exit system call to end program execution.

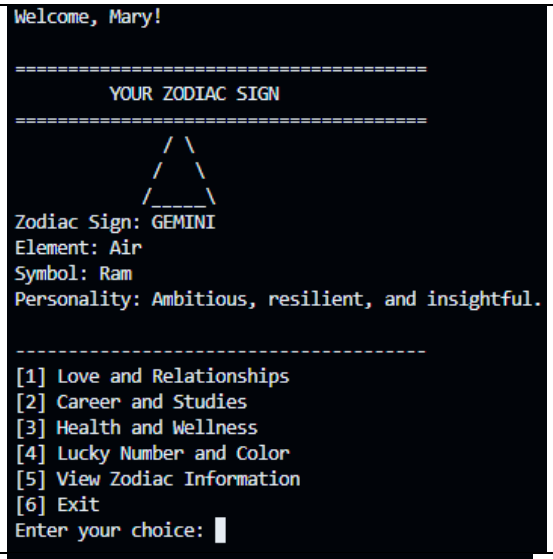
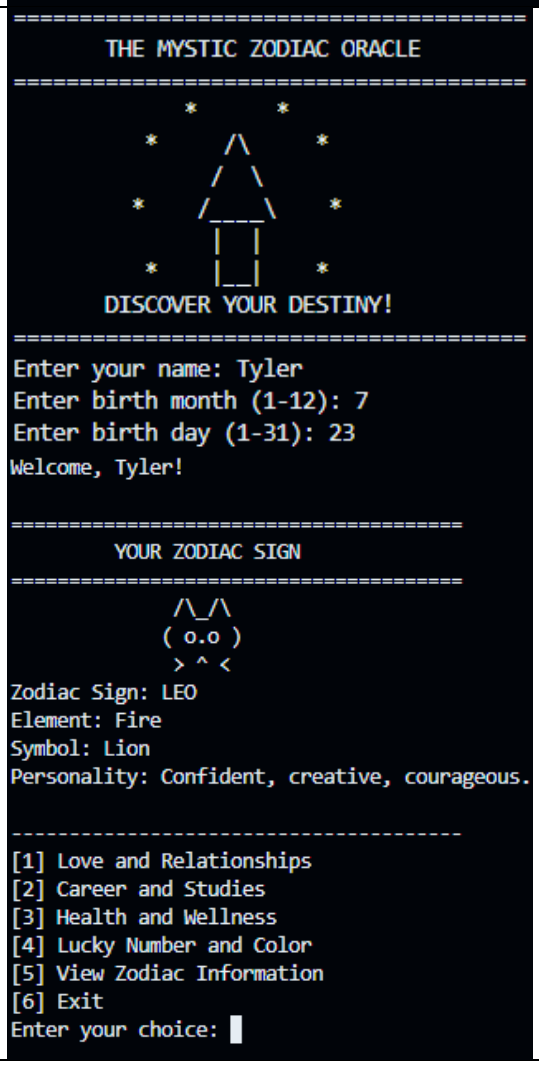
MACHINE PROBLEM 1: TEST CASES

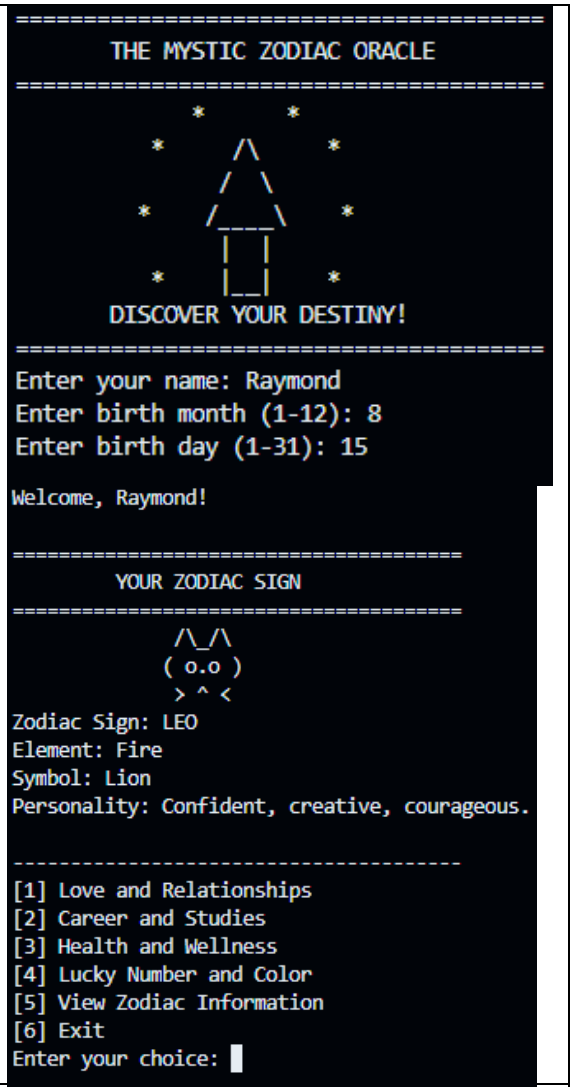
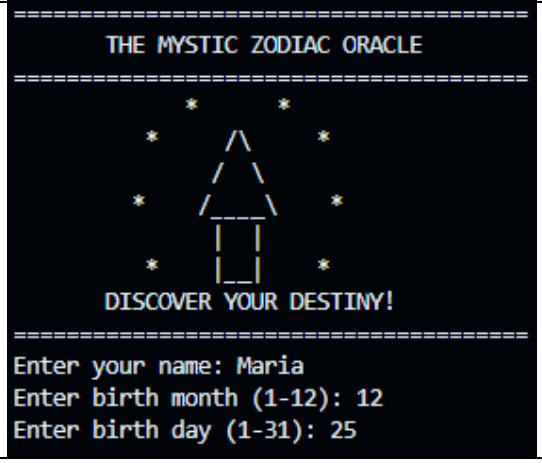


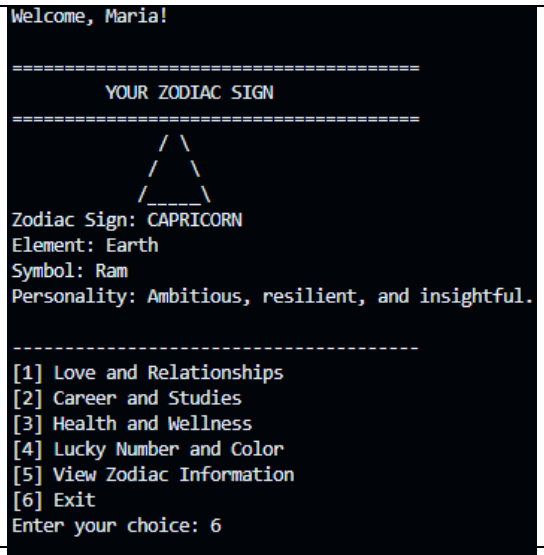
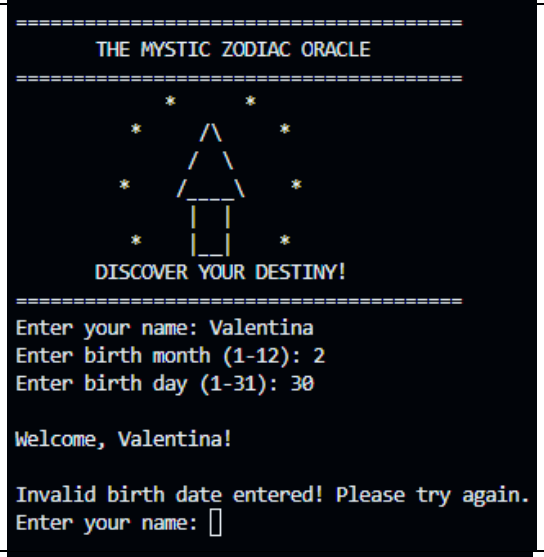
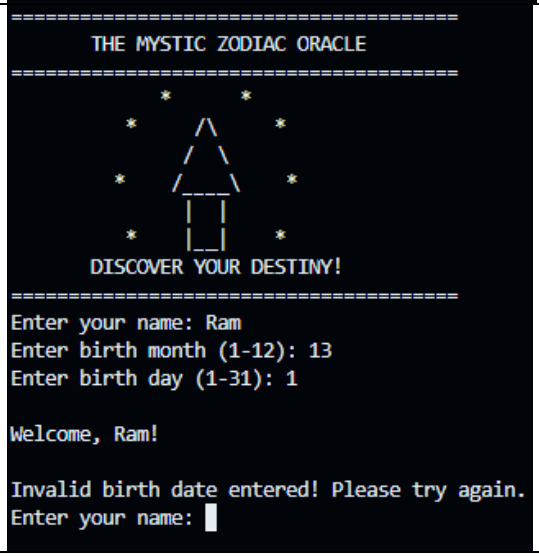
TEST CASE 1:
Input: March 21

	
TEST CASE 2: Input: April 19	

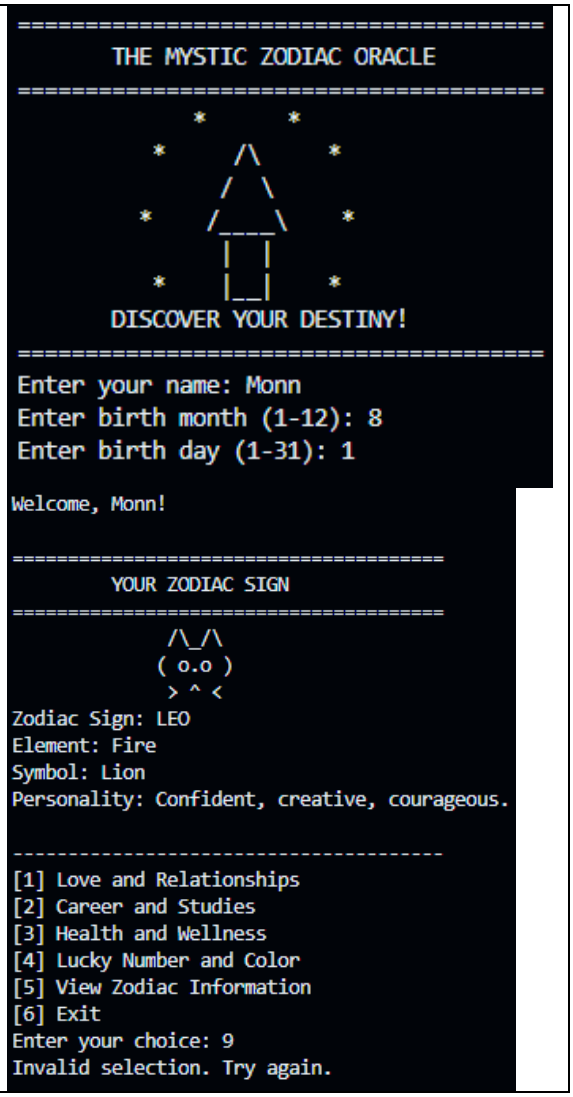
TEST CASE 3: Input: April 20	
TEST CASE 4: Input: June 15	

	
TEST CASE 5: Input: July 23	

TEST CASE 6: Input: August 15	
TEST CASE 7: Input: December 25	

	
TEST CASE 8: Input: February 30	
TEST CASE 9: Input: Month 13	

TEST CASE 10:
Input: Menu choice 9



Machine Problem 2 — Dungeon Adventure and Survival Game

Problem Statement

Create a text-based Dungeon Adventure and Survival Game. The player selects a class, explores three rooms, battles enemies, collects treasure, and makes decisions affecting survival. Use procedures, conditional selections, arithmetic, loops, user input, and ASCII art.

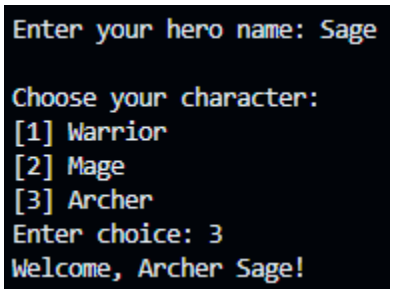
1. Welcome Screen

```
=======================================
       DUNGEON OF THE LOST KING
=======================================
               |>>>>
               |
           _  _|_  _
          |;|_|;|_|;|
          \.    .  /
           \:  .  /
            ||:   |
            ||:.  |
            ||:  .|
       ENTER IF YOU DARE!
=======================================
```



2. Player Registration

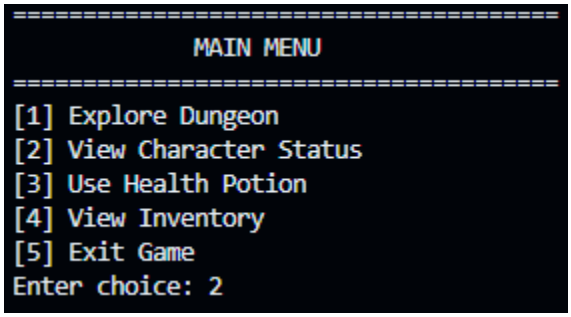
```
Enter your hero name: Arthur
Choose your character:
[1] Warrior
[2] Mage
[3] Archer
Enter choice: 1
Welcome, Warrior Arthur!
```



Character	Health	Attack	Potions
Warrior	100	20	2
Mage	70	30	3
Archer	85	25	2

3. Main Menu

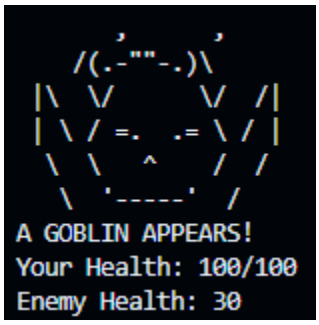
```
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
```



4. Dungeon Rooms and Gameplay

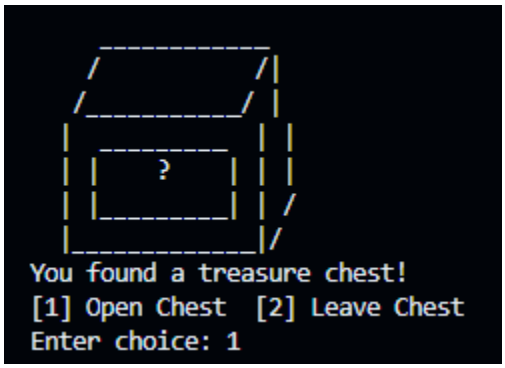
Room 1 — Goblin Chamber: Goblin health = 30. Display enemy ASCII art and offer Attack, Defend, Use Potion, or Run.

```
          ,      ,
         /(.-""-.)\
     |\  \/      \/  /|
     | \ / =.  .= \ / |
      \  \   ^    /  /
       \  '-----'  /
A GOBLIN APPEARS!
[1] Attack  [2] Defend
[3] Use Potion  [4] Run
```



Room 2 — Treasure Chamber: The player chooses to open the chest (gain 50 gold) or leave it (gain nothing).

```
          _________
         /_________/|
        |  _____  | |
        | |  ?  | | |
        | |_____| | /
        |_________|/
[1] Open Chest  [2] Leave Chest
```



Room 3 — Dragon's Lair: Dragon health = 60. Show dragon ASCII art and the same combat menu as the goblin.

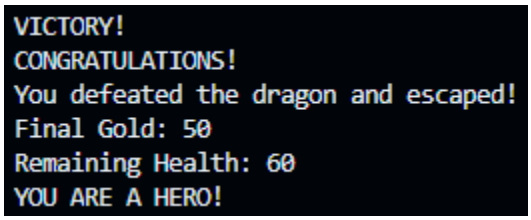
```
                / \  //\
      |\___/|      \
      /0  0  \       |
     /   ^    \     /
    /  \___/   \  /
    \___________\/
THE DRAGON AWAKENS!
```



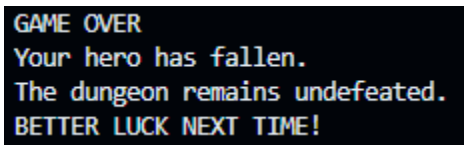
5. Combat and Outcome Rules

Attack: subtract player attack points from enemy health. Defend: reduce the next enemy attack by 5 (minimum damage 0). Potion: restore 20 health up to starting maximum. Run: leave encounter and return to menu without advancing. A surviving goblin deals 10 damage per turn; a surviving dragon deals 15. After Attack or Defend, the living enemy retaliates. A player loses at health 0 or less; victory occurs after defeating the dragon.

```
VICTORY!
CONGRATULATIONS, ARTHUR!
You defeated the dragon and escaped!
Final Gold: 50
Remaining Health: 35
YOU ARE A HERO!
```




```
GAME OVER
Your hero has fallen.
The dungeon remains undefeated.
BETTER LUCK NEXT TIME!
```



Required Procedures

Procedure	Responsibility
display_welcome	Show title and ASCII art
get_player_info	Read name and class
initialize_character	Assign stats
display_main_menu	Show options
explore_dungeon	Control room progression
battle_enemy	Handle combat selections
use_potion	Restore health
display_status	Show statistics
display_inventory	Show gold and potions
display_game_result	Show victory or defeat
exit_program	Terminate program

Validate class selection, menus, potion count, health boundaries, room progression, and win/loss conditions. Never allow negative displayed health or potions below zero.

Test Cases

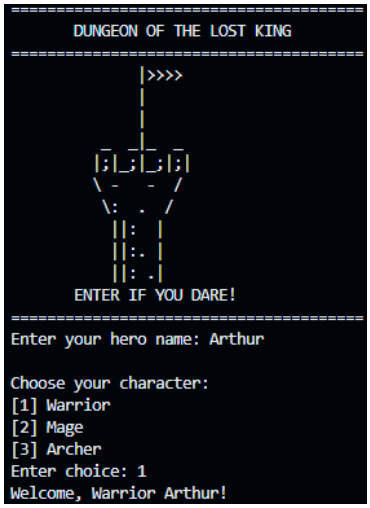
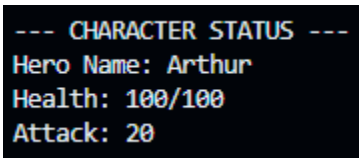
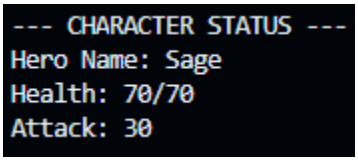
Test	Scenario	Expected result
1	Select Warrior	Health 100, Attack 20
2	Select Mage	Health 70, Attack 30
3	Select Archer	Health 85, Attack 25
4	Select class 9	Invalid selection
5	Warrior attacks Goblin twice	Goblin defeated
6	Potion at full health	Health capped; potion consumed
7	Potion count 0	Error; no health change
8	Open treasure	Gain 50 gold
9	Health reaches 0	Game Over
10	Dragon health reaches 0	Victory screen

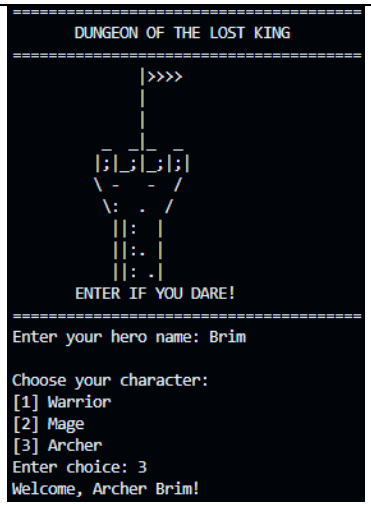
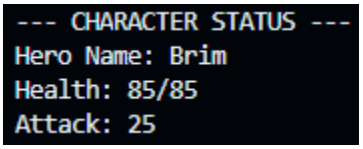
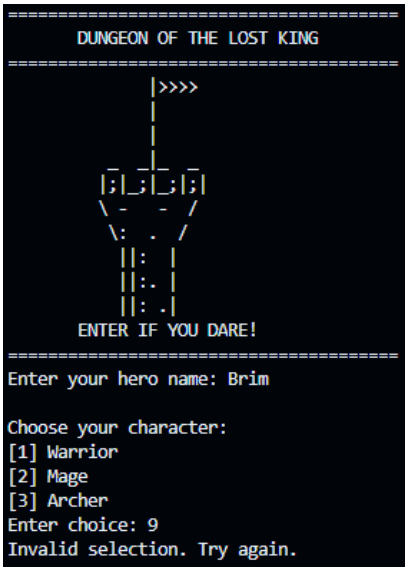
Explanation of Implemented Procedure

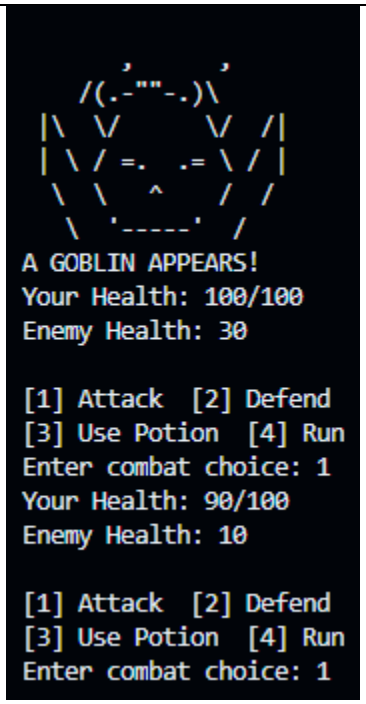
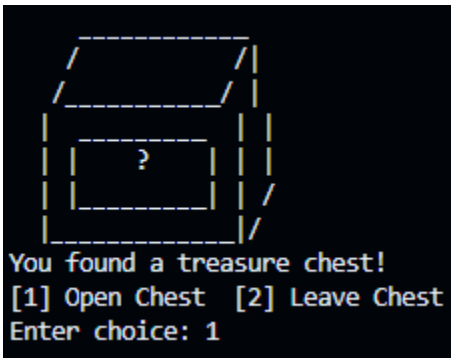
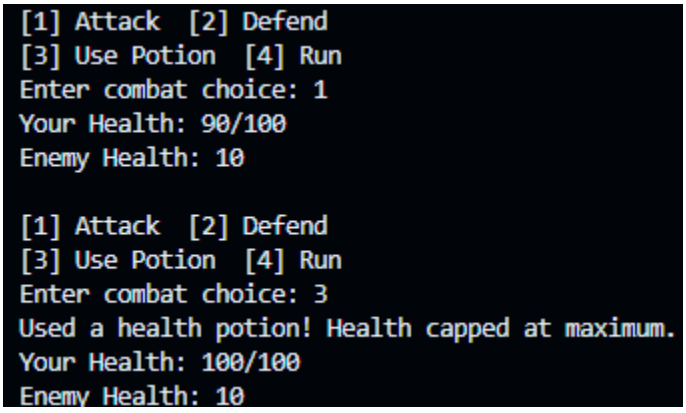
1. display_welcome Displays the game's welcome screen and castle-themed ASCII art. This procedure prints the title "DUNGEON OF THE LOST KING" and introduces the player to the adventure before any input is entered.
2. get_player_info Collects the player's hero name and character class selection. It reads the user's input, removes the trailing newline from the hero name, validates the class choice (Warrior, Mage, or Archer), and stores the selected class in player_class_id.
3. initialize_character Initializes the player's starting attributes based on the chosen class. Warrior receives 100 HP, 20 ATK, and 2 potions, Mage receives 70 HP, 30 ATK, and 3 potions, and Archer receives 85 HP, 25 ATK, and 2 potions. This procedure also sets the starting gold, current room, and defensive status.
4. display_main_menu Displays the main game menu. It provides options to explore the dungeon, view character status, use a health potion, view inventory, or exit the game.
5. explore_dungeon Controls dungeon progression by checking the value stored in current_room. Based on the current room, it sends the player to the Goblin Chamber, Treasure Chamber, Dragon's Lair, or displays a dungeon completion message if the game has already been finished.
6. battle_enemy Handles the entire combat system. It displays player and enemy health, processes combat actions such as Attack, Defend, Use Potion, and Run, applies enemy damage, and checks whether the player or enemy has been defeated before continuing the battle loop.
7. use_potion Manages potion consumption and health recovery. The procedure first checks whether potions are available, decreases the potion count when used, restores 20 health points, and ensures that health does not exceed the character's maximum HP.
8. display_status Displays the player's current character information. It shows the hero's name, current and maximum health, and attack power so the player can monitor their progress.
9. display_inventory Displays the contents of the player's inventory. It shows the number of remaining health potions and the amount of gold collected throughout the adventure.
10. display_game_result Displays the final outcome of the game. If the player survives and defeats the dragon, it shows the victory screen, remaining health, and final gold. If the player's health reaches zero, it displays the game over message instead.
11. exit_program Terminates the game using the Linux exit system call. This procedure is executed when the player chooses to quit the game or when a victory or defeat condition

ends the adventure.

MACHINE PROBLEM 2: TEST CASES

TEST CASE 1: Select Warrior OUTPUT: --- CHARACTER STATUS --- Hero Name: Arthur Health: 100/100 Attack: 20	 
TEST CASE 2: Select Mage OUTPUT: --- CHARACTER STATUS --- Hero Name: Sage Health: 70/70 Attack: 30	 
TEST CASE 3: Select Archer OUTPUT: --- CHARACTER STATUS ---	

Hero Name: Brim Health: 85/85 Attack: 25	 
TEST CASE 4: Select Class 9 OUTPUT: Invalid selection. Try again.	

TEST CASE 5: Warrior attacks Goblin twice OUTPUT: Goblin Defeated! You found a treasure chest!	 
TEST CASE 6: Potion at Full Health OUTPUT: Used a health potion! Health capped at maximum. Your Health: 100/100 Enemy Health: 10	
TEST CASE 7: Potion count 0 OUTPUT: Error; no health change (No potions remaining!) Your Health: 100/100	

Enemy Health: 10	[1] Attack [2] Defend [3] Use Potion [4] Run Enter combat choice: 3 Error; no health change (No potions remaining!) Your Health: 100/100 Enemy Health: 10
TEST CASE 8: Open Treasure OUTPUT:	 You found a treasure chest! [1] Open Chest [2] Leave Chest Enter choice: 1 Gained 50 gold!
TEST CASE 9: Health reaches 0 OUTPUT: GAME OVER Your hero has fallen. The dungeon remains undefeated. BETTER LUCK NEXT TIME!	GAME OVER Your hero has fallen. The dungeon remains undefeated. BETTER LUCK NEXT TIME!
TEST CASE 10: Dragon health reaches 0 OUTPUT: VICTORY! CONGRATULATIONS! You defeated the dragon and escaped! Final Gold: 50 Remaining Health: 70 YOU ARE A HERO!	VICTORY! CONGRATULATIONS! You defeated the dragon and escaped! Final Gold: 50 Remaining Health: 70 YOU ARE A HERO!

Assessment Rubric

Criterion	Points
Correct procedures (CALL/RET)	25
Conditional selection and branching	20
User input and validation	15
Correct logic and output	15
ASCII art and interface creativity	10
Looping and menu navigation	10
Readability and comments	5
TOTAL	100

Required Student Deliverables

Submit the .asm source file, screenshots of successful execution and invalid-input handling, a short demonstration video, and a brief explanation of implemented procedures.

Open-Ended Questions

1. **What is the purpose of procedures in your application?**

Procedures break complex program tasks into smaller, reusable, and self-contained routines, improving code readability and organization.

2. **How do CALL and RET control program execution?**

CALL pushes the return address onto the call stack and branches execution to a procedure, RET pops that saved address back into EIP to resume execution right after the CALL.

3. **Which conditional jump instructions did you use, and why?**

JE and JNE were used for exact menu option matches, while JL, JG, JLE, and JGE handled numerical boundary checks for dates, health ranges, and damage calculations.

4. **How does your program handle invalid user input?**

It validates input values immediately against allowed constraints, displays an error message (msg_invalid) when validation fails, and prompts the user again.

5. **What would be harder to maintain without procedures?**

Debugging, code reuse, and structural flow would be extremely difficult because all menu loops, I/O routines, and game state logic would be mixed together in one long block.

Application-Specific Open-Ended Questions

Answer all questions for the application you developed. Write 3–5 complete sentences per question, refer to specific procedures or instructions from your own source code, and provide examples where requested.

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

1. **Explain how your determine_zodiac procedure identifies the correct sign from the birth month and day. Describe at least two conditional comparisons.**

It checks birth_month using CMP eax, <month> and branches via JE. Inside each month block, it uses CMP ebx, <cutoff_day> and JLE to select the early-month sign or jumps to the late-month sign.

2. **How does your program handle zodiac date ranges that cross two months, such as Capricorn? Explain the branching logic.**

It checks both months separately: if month is 12 and day is JG 21, it sets Capricorn; if month is 1 and day is JLE 19, it also sets Capricorn.

3. **Why did you divide input collection, date validation, zodiac identification, and horoscope display into separate procedures?**

Dividing tasks isolates I/O processing, validation logic, sign logic, and UI display into independent units, preventing code duplication and making maintenance straightforward.

4. **Describe how your program rejects invalid dates such as February 30 and an invalid month such as 13.**

validate_birthdate checks if month is outside 1–12 or day is outside 1–31. A targeted check (check_feb) rejects any February day greater than 28, returning 0 in EAX to signal invalid input.

5. **How does the program select a horoscope category after the user enters a menu choice? Identify the conditional jump instructions used.**

It converts the selection byte to an integer and evaluates it with CMP instructions, branching using JE show_love, JE show_career, JE show_health, JE show_lucky, or JE show_zodiac_info.

6. **How did you implement the zodiac ASCII art? Explain how the correct design is selected and displayed.**

ASCII designs are stored in .data with newline characters (0x0A). display_zodiac_art evaluates zodiac_id using CMP and JE print_leo_art to load the appropriate string address into ECX for sys_write (int 0x80).

7. **What happens when a user enters an invalid menu option? Explain how control returns to the menu.**

Selections outside the valid option range fail the CMP checks and jump to invalid_menu_opt, which prints msg_invalid and jumps back to main_menu_loop.

8. **If you added a new horoscope category, which procedures and conditional branches would you modify? Explain why.**

You would update display_menu to list the new option, adjust the upper bound check in main_menu_loop (CMP), and add a new CMP/JE branch in show_horoscope to call the new display procedure.

Machine Problem 2 — Dungeon Adventure and Survival Game

1. **Explain how initialize_character assigns different health, attack, and potion values for Warrior, Mage, and Archer.**

In my program, the initialize_character procedure checks the value stored in player_class_id and uses cmp and je instructions to jump to the correct class initialization section. If the player chooses Warrior, the program assigns 100 HP, 20 attack, and 2 potions. If the player chooses Mage, it assigns 70 HP, 30 attack, and 3 potions, while Archer receives 85 HP, 25 attack, and 2 potions. This implementation makes each class play differently because the Mage has the highest attack, the Warrior has the highest health, and the Archer is balanced between the two.

2. **Describe the procedures and conditional jumps used when the player chooses Attack, Defend, Use Potion, or Run.**

Inside the battle_enemy procedure, the player's choice is read and compared using a series of cmp instructions. When the player selects Attack, the program jumps to cmd_attack, subtracts player_atk from enemy_hp, and then proceeds to enemy_turn. When Defend is selected, it jumps to cmd_defend, sets the defending flag to 1, and then continues to the enemy's turn. Choosing Use Potion calls the use_potion procedure and returns to the combat loop, while selecting Run jumps to cmd_run, which immediately returns from the battle procedure.

3. **How does your battle_enemy procedure determine whether the player or enemy has been defeated? Explain the comparisons.**

At the start of every combat loop, our program checks both the enemy's and player's health values. It compares enemy_hp with zero using cmp eax, 0 and jumps to combat_won when the value is less than or equal to zero using jle. It also checks player_hp in the same way and jumps to combat_lost if the player's health reaches zero or below. These comparisons ensure that combat ends immediately whenever either side has no remaining health.

4. **Explain how you prevent health from exceeding the maximum and prevent potion use when the inventory is empty.**

The use_potion procedure first checks the number of available potions stored in player_pots. If the value is zero or less, the jle no_potions jump is executed and an error message is displayed instead of healing the player. When a potion is successfully used, the program adds 20 to player_hp and then compares the result with player_max_hp. If the current health becomes greater than the maximum, the program overwrites player_hp with player_max_hp, preventing health from exceeding the allowed limit.

5. **How is the current dungeon room tracked, and what conditions allow the player to advance to the next room?**

My program tracks dungeon progress using the variable current_room. The explore_dungeon procedure checks this variable and jumps to the corresponding room procedure such as room_1_goblin, room_2_treasure, or room_3_dragon. After defeating the goblin, the program checks if enemy_hp is less than or equal to zero and then sets current_room to 1. The treasure room automatically changes current_room to 2, while defeating the dragon sets it to 3, indicating that the dungeon has been completed.

6. **What is the purpose of separating display_status, display_inventory, and battle_enemy into different procedures?**

I separated these features into different procedures to make the program more organized and easier to manage. The display_status procedure is responsible only for showing the hero's statistics, while display_inventory handles potions and gold information. The battle_enemy procedure contains all combat-related logic such as attacking, defending, taking damage, and checking victory conditions. By separating them, I can call each feature whenever needed without rewriting the same code multiple times.

7. **How does your program preserve important register values across procedures and Linux system calls? Give one example.**

One example is the print_number procedure, where I use the pusha instruction at the beginning and popa before returning. These instructions save and restore all general-purpose registers, preventing the procedure from permanently changing values that other parts of the program may still need. This is important because print_number performs several calculations and Linux write system calls while converting integers into printable characters. Using pusha and popa ensures that execution returns with the original register values intact.

8. **If you introduced a fourth room with a stronger enemy, what program data, procedures, and conditions would need to change?**

If I added a fourth room, I would first create new data such as the enemy artwork, messages, and a stronger enemy health value in the .data section. I would then add a new room procedure similar to room_3_dragon and update explore_dungeon so that a new current_room value points to that room. The room progression conditions would also need to be adjusted so that defeating the dragon advances the player to Room 4 instead of ending the game. Finally, the victory condition would be moved to the new room, and defeating the stronger enemy there would set the final completion state and display the game results.

Submission Requirement for Written Responses

Include a separate "Open-Ended Questions" section in your submission. Answer the five general questions and the eight questions for your assigned machine problem. Answers must reflect your actual program implementation; unsupported or generic explanations will not receive full credit.

Instructor Guidance

Assign Zodiac Horoscope first to focus on procedures, branching, and input validation. Assign Dungeon Adventure afterward for arithmetic, game state, repeated decisions, and interacting procedures.

Open-Ended Questions:

1. **What is the purpose of procedures in your application?**

Procedures break complex program tasks into smaller, reusable, and self-contained routines, improving code readability and organization.

2. **How do CALL and RET control program execution?**

CALL pushes the return address onto the call stack and branches execution to a procedure, RET pops that saved address back into EIP to resume execution right after the CALL.

3. **Which conditional jump instructions did you use, and why?**

JE and JNE were used for exact menu option matches, while JL, JG, JLE, and JGE handled numerical boundary checks for dates, health ranges, and damage calculations.

4. **How does your program handle invalid user input?**

It validates input values immediately against allowed constraints, displays an error message (msg_invalid) when validation fails, and prompts the user again.

5. What would be harder to maintain without procedures?

Debugging, code reuse, and structural flow would be extremely difficult because all menu loops, I/O routines, and game state logic would be mixed together in one long block.